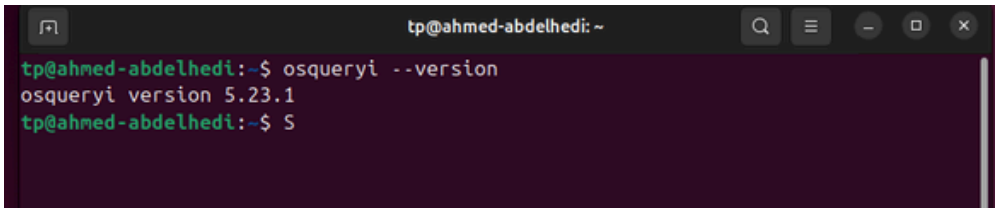
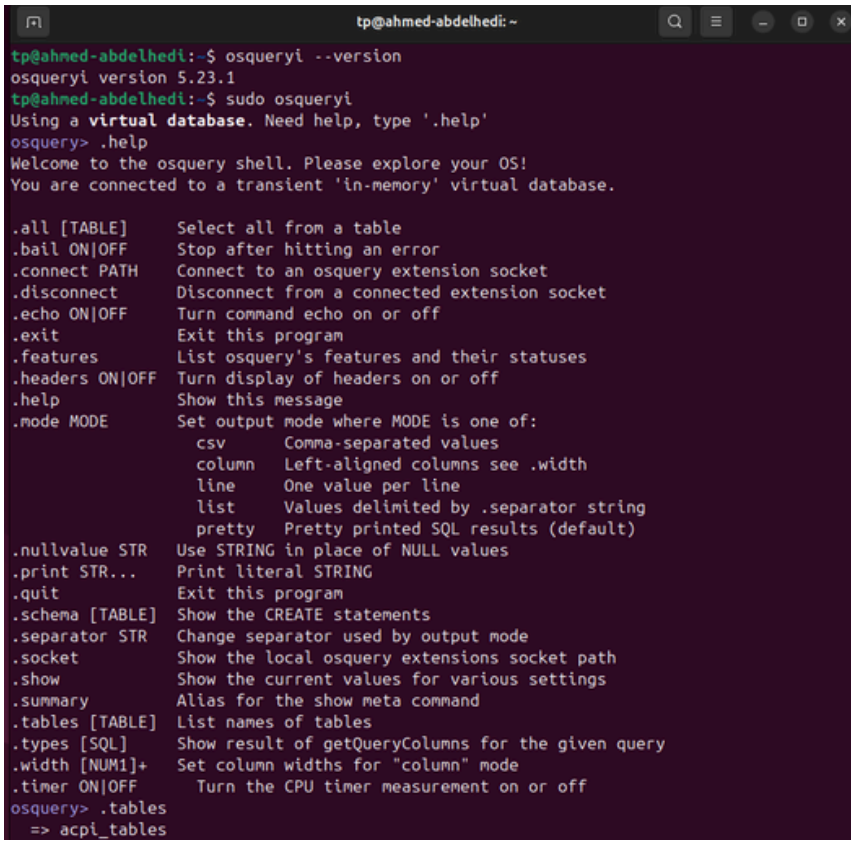
Endpoint Data Collection and Observation using osquery

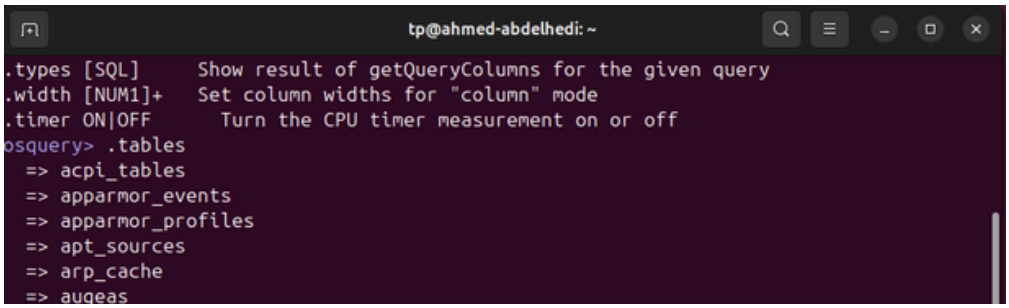
Ahmed ABDELHEDI

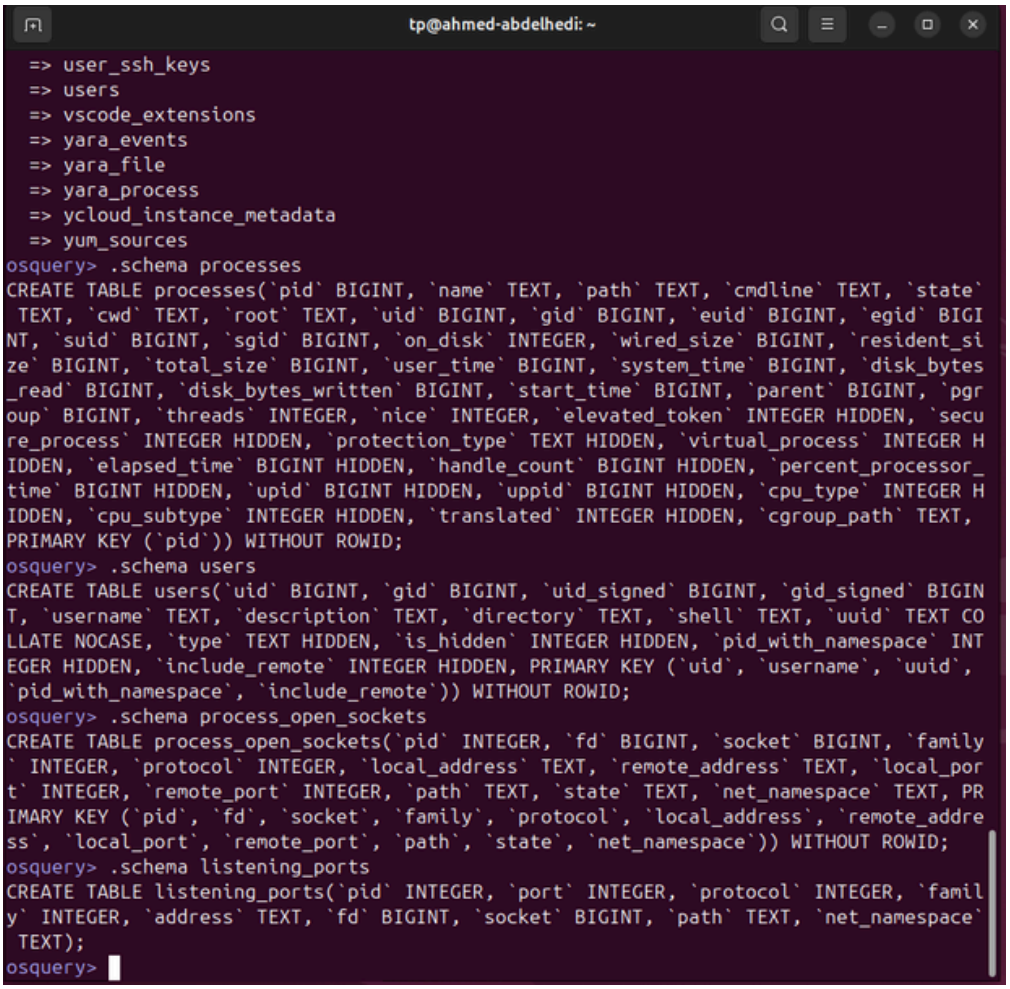
II. INSTALLING OSQUERY



III. EXPLORING THE OSQUERY INTERACTIVE SHELL

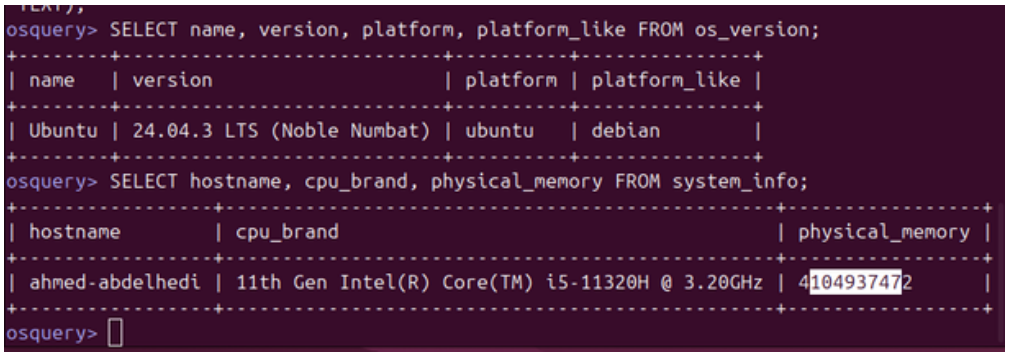



tp@ahmed-abdelhedi: ~
.types [SQL]   Show result of getQueryColumns for the given query
.width [NUM1]+   Set column widths for "column" mode
.timer ON|OFF   Turn the CPU timer measurement on or off
osquery> .tables
  => acpi_tables
  => apparmor_events
  => apparmor_profiles
  => apt_sources
  => arp_cache
  => augeas


tp@ahmed-abdelhedi: ~
  => user_ssh_keys
  => users
  => vscode_extensions
  => yara_events
  => yara_file
  => yara_process
  => ycloud_instance_metadata
  => yum_sources
osquery> .schema processes
CREATE TABLE processes(`pid` BIGINT, `name` TEXT, `path` TEXT, `cmdline` TEXT, `state`
 TEXT, `cwd` TEXT, `root` TEXT, `uid` BIGINT, `gid` BIGINT, `euid` BIGINT, `egid` BIGI
NT, `suid` BIGINT, `sgid` BIGINT, `on_disk` INTEGER, `wired_size` BIGINT, `resident_si
ze` BIGINT, `total_size` BIGINT, `user_time` BIGINT, `system_time` BIGINT, `disk_bytes
_read` BIGINT, `disk_bytes_written` BIGINT, `start_time` BIGINT, `parent` BIGINT, `pgr
oup` BIGINT, `threads` INTEGER, `nice` INTEGER, `elevated_token` INTEGER HIDDEN, `secu
re_process` INTEGER HIDDEN, `protection_type` TEXT HIDDEN, `virtual_process` INTEGER H
IDDEN, `elapsed_time` BIGINT HIDDEN, `handle_count` BIGINT HIDDEN, `percent_processor_
time` BIGINT HIDDEN, `upid` BIGINT HIDDEN, `uppid` BIGINT HIDDEN, `cpu_type` INTEGER H
IDDEN, `cpu_subtype` INTEGER HIDDEN, `translated` INTEGER HIDDEN, `cgroup_path` TEXT,
PRIMARY KEY (`pid`)) WITHOUT ROWID;
osquery> .schema users
CREATE TABLE users(`uid` BIGINT, `gid` BIGINT, `uid_signed` BIGINT, `gid_signed` BIGIN
T, `username` TEXT, `description` TEXT, `directory` TEXT, `shell` TEXT, `uuid` TEXT CO
LLATE NOCASE, `type` TEXT HIDDEN, `is_hidden` INTEGER HIDDEN, `pid_with_namespace` INT
EGER HIDDEN, `include_remote` INTEGER HIDDEN, PRIMARY KEY (`uid`, `username`, `uuid`,
`pid_with_namespace`, `include_remote`)) WITHOUT ROWID;
osquery> .schema process_open_sockets
CREATE TABLE process_open_sockets(`pid` INTEGER, `fd` BIGINT, `socket` BIGINT, `family
` INTEGER, `protocol` INTEGER, `local_address` TEXT, `remote_address` TEXT, `local_por
t` INTEGER, `remote_port` INTEGER, `path` TEXT, `state` TEXT, `net_namespace` TEXT, PR
IMARY KEY (`pid`, `fd`, `socket`, `family`, `protocol`, `local_address`, `remote_addre
ss`, `local_port`, `remote_port`, `path`, `state`, `net_namespace`)) WITHOUT ROWID;
osquery> .schema listening_ports
CREATE TABLE listening_ports(`pid` INTEGER, `port` INTEGER, `protocol` INTEGER, `famil
y` INTEGER, `address` TEXT, `fd` BIGINT, `socket` BIGINT, `path` TEXT, `net_namespace`
 TEXT);
osquery>

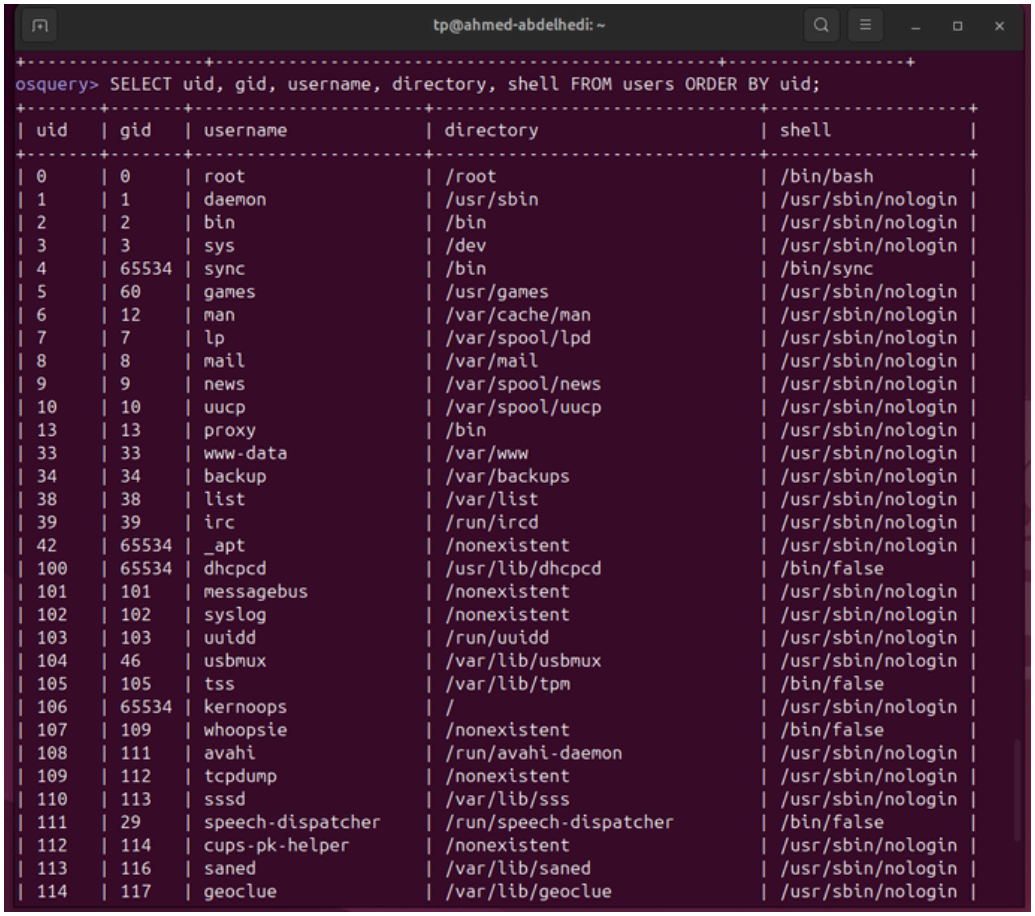
IV. CHOOSING AND COLLECTING ENDPOINT OBSERVABLES

System context



```
osquery> SELECT name, version, platform, platform_like FROM os_version;
+--------+----------------------------+----------+---------------+
| name   | version                    | platform | platform_like |
+--------+----------------------------+----------+---------------+
| Ubuntu | 24.04.3 LTS (Noble Numbat) | ubuntu   | debian        |
+--------+----------------------------+----------+---------------+
osquery> SELECT hostname, cpu_brand, physical_memory FROM system_info;
+----------------+-----------------------------------------------+-----------------+
| hostname       | cpu_brand                                     | physical_memory |
+----------------+-----------------------------------------------+-----------------+
| ahmed-abdelhedi | 11th Gen Intel(R) Core(TM) i5-11320H @ 3.20GHz | 4104937472      |
+----------------+-----------------------------------------------+-----------------+
osquery>
```

Users

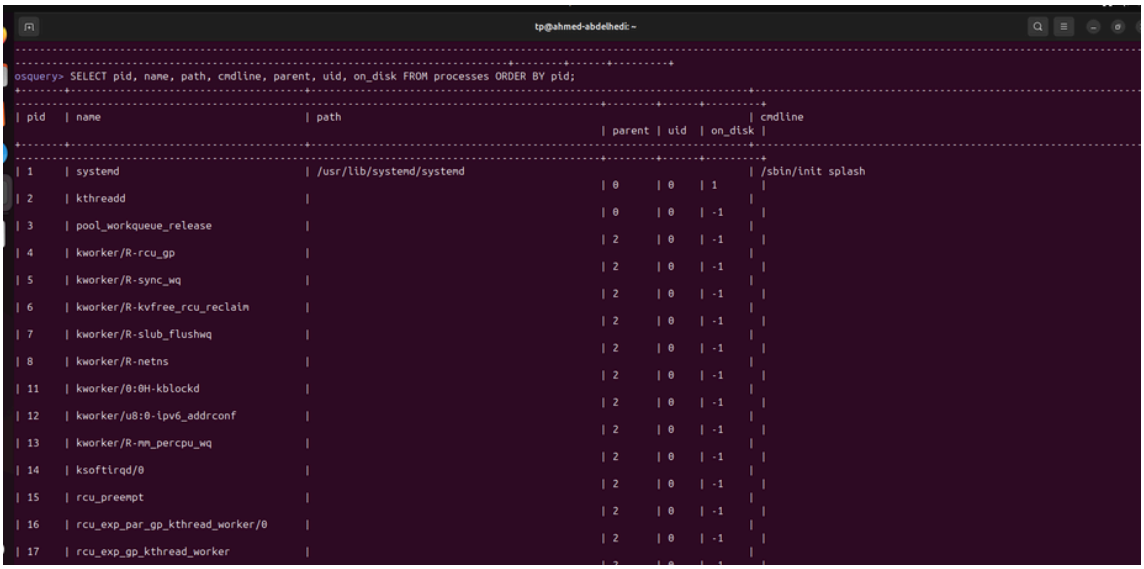


```
tp@ahmed-abdelhedi: ~
osquery> SELECT uid, gid, username, directory, shell FROM users ORDER BY uid;
+-------+-------+--------------------+------------------------------+------------------+
| uid   | gid   | username           | directory                    | shell            |
+-------+-------+--------------------+------------------------------+------------------+
| 0     | 0     | root               | /root                        | /bin/bash        |
| 1     | 1     | daemon             | /usr/sbin                    | /usr/sbin/nologin |
| 2     | 2     | bin                | /bin                         | /usr/sbin/nologin |
| 3     | 3     | sys                | /dev                         | /usr/sbin/nologin |
| 4     | 65534 | sync               | /bin                         | /bin/sync        |
| 5     | 60    | games              | /usr/games                   | /usr/sbin/nologin |
| 6     | 12    | man                | /var/cache/man               | /usr/sbin/nologin |
| 7     | 7     | lp                 | /var/spool/lpd               | /usr/sbin/nologin |
| 8     | 8     | mail               | /var/mail                    | /usr/sbin/nologin |
| 9     | 9     | news               | /var/spool/news              | /usr/sbin/nologin |
| 10    | 10    | uucp               | /var/spool/uucp              | /usr/sbin/nologin |
| 13    | 13    | proxy              | /bin                         | /usr/sbin/nologin |
| 33    | 33    | www-data           | /var/www                     | /usr/sbin/nologin |
| 34    | 34    | backup             | /var/backups                 | /usr/sbin/nologin |
| 38    | 38    | list               | /var/list                    | /usr/sbin/nologin |
| 39    | 39    | irc                | /run/ircd                    | /usr/sbin/nologin |
| 42    | 65534 | _apt               | /nonexistent                 | /usr/sbin/nologin |
| 100   | 65534 | dhcpcd             | /usr/lib/dhcpcd              | /bin/false       |
| 101   | 101   | messagebus         | /nonexistent                 | /usr/sbin/nologin |
| 102   | 102   | syslog             | /nonexistent                 | /usr/sbin/nologin |
| 103   | 103   | uuidd              | /run/uuidd                   | /usr/sbin/nologin |
| 104   | 46    | usbmux             | /var/lib/usbmux              | /usr/sbin/nologin |
| 105   | 105   | tss                | /var/lib/tpm                 | /bin/false       |
| 106   | 65534 | kernoops           | /                            | /usr/sbin/nologin |
| 107   | 109   | whoopsie           | /nonexistent                 | /bin/false       |
| 108   | 111   | avahi              | /run/avahi-daemon            | /usr/sbin/nologin |
| 109   | 112   | tcpdump            | /nonexistent                 | /usr/sbin/nologin |
| 110   | 113   | sssd               | /var/lib/sss                 | /usr/sbin/nologin |
| 111   | 29    | speech-dispatcher  | /run/speech-dispatcher       | /bin/false       |
| 112   | 114   | cups-pk-helper     | /nonexistent                 | /usr/sbin/nologin |
| 113   | 116   | saned              | /var/lib/saned               | /usr/sbin/nologin |
| 114   | 117   | geoclue            | /var/lib/geoclue             | /usr/sbin/nologin |
```

11. Identify the local account with UID 0. Report its username, home directory, and configured login shell

The account with UID 0 is root. Its home directory is /root, and its login shell is /bin/bash.

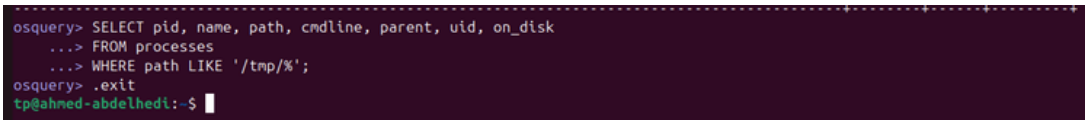
Processes



Which of these attributes could be useful for detecting suspicious process execution? Briefly explain why.

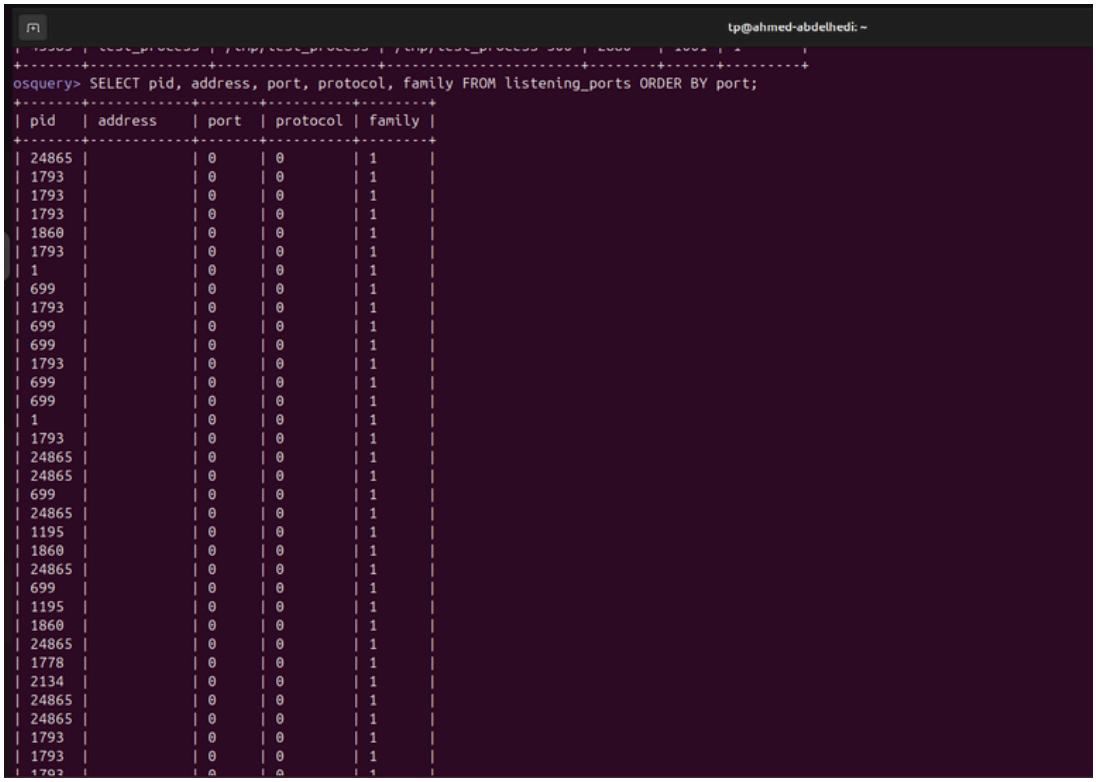
The most useful attributes are path, cmdline, and parent. They reveal whether a process runs from a suspicious location, executes unusual commands, or was launched by an unexpected process.

13.





Network activity



We use the second table, processes, to obtain the executable path of the process responsible for each network connection by matching their PIDs.



NetworkManager (PID 812) uses UDP over IPv4 to communicate from 10.0.2.15:68 to the DHCP server at 10.0.2.2:67. This is legitimate DHCP traffic used to configure the VM's network connection.

Which fields in the output could be useful as features for intrusion detection? Can a remote network connection alone be considered evidence of malicious activity?

The remote address, remote port, protocol, and process path are useful intrusion-detection features. A remote connection alone is not evidence of malicious activity because legitimate applications also communicate with remote systems. Its context and the responsible process must also be examined

Files and hashes

16:

```
osquery> .exit
[1]+  Done                    /tmp/test_process 300
tp@ahmed-abdelhedi:~$ echo 'osquery lab' > /tmp/testfile.txt
tp@ahmed-abdelhedi:~$ sudo osqueryi
[sudo] password for tp:
Using a virtual database. Need help, type '.help'
osquery> SELECT path, filename, size, mode, uid, gid, mtime FROM file WHERE path =
    ...> '/tmp/testfile.txt';
+------------------+-------------+------+------+------+------+------------+
| path             | filename    | size | mode | uid  | gid  | mtime      |
+------------------+-------------+------+------+------+------+------------+
| /tmp/testfile.txt | testfile.txt | 12   | 0664 | 1001 | 1001 | 1789301719 |
+------------------+-------------+------+------+------+------+------------+
osquery>
```

The file /tmp/testfile.txt has a size of 12 bytes, permissions 0664, and belongs to user and group ID 1001.

17:

```
osquery> SELECT path, sha256
    ...> FROM hash
    ...> WHERE path = '/tmp/testfile.txt';
+------------------+------------------------------------------------------------------+
| path             | sha256                                                           |
+------------------+------------------------------------------------------------------+
| /tmp/testfile.txt | 621dc6410401957fd620cb46a8f29165c24bf399a5d6726dfe080b15f8543e1d |
+------------------+------------------------------------------------------------------+
osquery>
```

It uniquely identifies the file's content and changes if the file is modified.

V. CORRELATING OBSERVATIONS: FROM TABLES TO SECURITY CONTEXT

18:

```
tp@ahmed-abdelhedi: ~
osquery> SELECT
    ...>     l.address,
    ...>     l.port,
    ...>     l.protocol,
    ...>     l.pid,
    ...>     p.name,
    ...>     p.path,
    ...>     p.cmdline,
    ...>     p.uid
    ...> FROM listening_ports AS l
    ...> JOIN processes AS p ON l.pid = p.pid
    ...> WHERE l.port != 0
    ...> ORDER BY l.port;
 address = 0.0.0.0
    port = 22
protocol = 6
     pid = 1
    name = systemd
    path = /usr/lib/systemd/systemd
 cmdline = /usr/lib/systemd/systemd --system --deserialize=92 splash
     uid = 0

 address = ::
    port = 22
protocol = 6
     pid = 1
    name = systemd
    path = /usr/lib/systemd/systemd
 cmdline = /usr/lib/systemd/systemd --system --deserialize=92 splash
     uid = 0

 address = 127.0.0.54
    port = 53
protocol = 6
     pid = 25608
    name = systemd-resolve
    path = /usr/lib/systemd/systemd-resolved
 cmdline = /usr/lib/systemd/systemd-resolved
     uid = 991
```

19:

The tuple {process path, command line, user, listening port} combines:
where the program is located;
how it was launched;
which user runs it;
which network port it exposes.

20:

Combining these attributes provides stronger context because one normal-looking value may become suspicious when related to the others. For example, Python is legitimate, but a Python process running from /tmp, executed by root, and listening on an unusual port such as 4444 could require investigation.

VI. CONTROLLED ENDPOINT INVESTIGATION SCENARIO

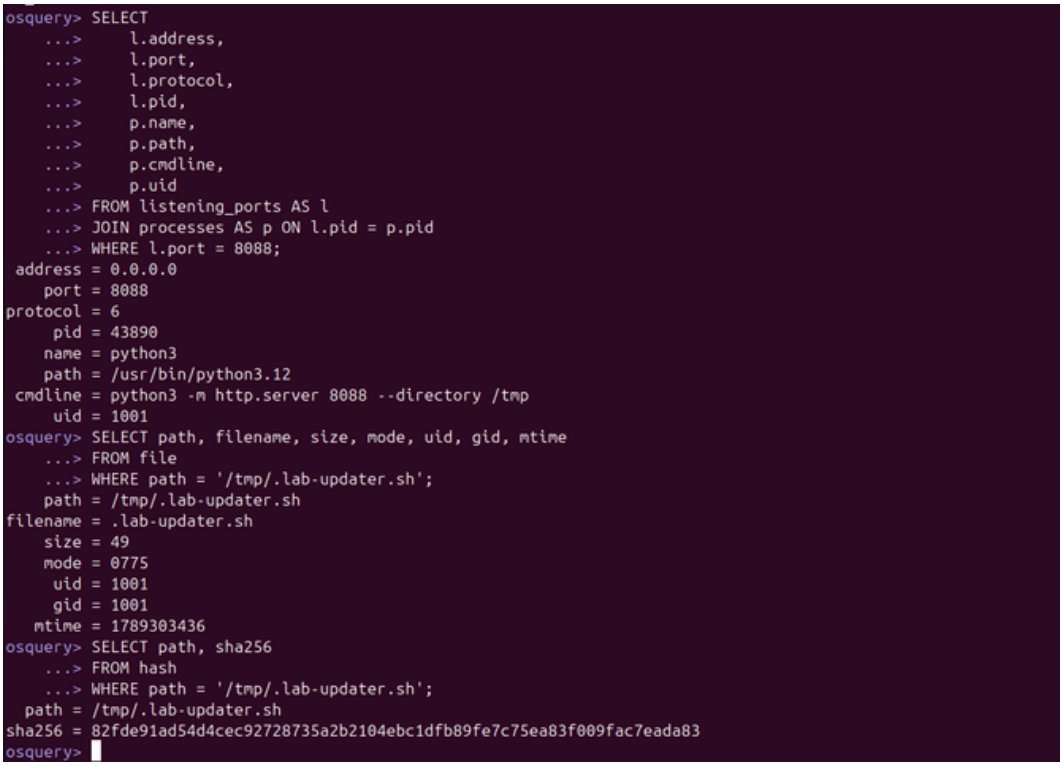
21:

```
osquery> .exit
tp@ahmed-abdelhedi:~$ cat > /tmp/.lab-updater.sh <<'EOF'
#!/bin/bash
echo "lab updater running"
sleep 600
EOF
tp@ahmed-abdelhedi:~$ chmod +x /tmp/.lab-updater.sh
tp@ahmed-abdelhedi:~$ ls  /tmp/.lab-updater.sh
/tmp/.lab-updater.sh
tp@ahmed-abdelhedi:~$ cat  /tmp/.lab-updater.sh
#!/bin/bash
echo "lab updater running"
sleep 600
tp@ahmed-abdelhedi:~$
```

22+23:

```
osquery> .mode line
osquery> SELECT pid, name, path, cmdline, parent, uid, on_disk
   ...> FROM processes
   ...> WHERE cmdline LIKE '%lab-updater%'
   ...>    OR cmdline LIKE '%http.server 8088%';
    pid = 43888
   name = .lab-updater.sh
   path = /usr/bin/bash
cmdline = /bin/bash /tmp/.lab-updater.sh
 parent = 2880
    uid = 1001
on_disk = 1

    pid = 43890
   name = python3
   path = /usr/bin/python3.12
cmdline = python3 -m http.server 8088 --directory /tmp
 parent = 2880
    uid = 1001
on_disk = 1
osquery> SELECT
   ...>     l.address,
   ...>     l.port,
   ...>     l.protocol,
   ...>     l.pid,
   ...>     p.name,
   ...>     p.path,
   ...>     p.cmdline,
   ...>     p.uid
   ...> FROM listening_ports AS l
   ...> JOIN processes AS p ON l.pid = p.pid
   ...> WHERE l.port = 8088;
 address = 0.0.0.0
    port = 8088
protocol = 6
     pid = 43890
    name = python3
    path = /usr/bin/python3.12
 cmdline = python3 -m http.server 8088 --directory /tmp
```



Process execution. The process table confirms that the hidden script .lab-updater.sh executes through Bash with PID 43888. It also identifies a Python HTTP server running with PID 43890. Both processes belong to UID 1001.

Network exposure. The listening-port query links TCP port 8088 to the Python process through PID 43890. The address 0.0.0.0 indicates that the server listens on all IPv4 interfaces, while its command line shows that it serves the /tmp directory.

File evidence. The file table confirms that /tmp/.lab-updater.sh exists on disk. It has a size of 49 bytes, executable permissions (0775), and belongs to UID and GID 1001.

File identification. The script's SHA-256 hash is:
82fde91ad54d4cec92728735a2b2104ebc1dfb89fe7c75ea83f009fac7eada83

Unlike a path or filename, the hash identifies the file's exact content and changes when the file is modified. Combining the process, network, file and ownership observations provides stronger context for characterizing the activity.

24:

```
tp@ahmed-abdelhedi:~$ pkill -f '/tmp/.lab-updater.sh'
tp@ahmed-abdelhedi:~$ pkill -f 'http.server 8088'
tp@ahmed-abdelhedi:~$ rm -f /tmp/.lab-updater.sh /tmp/lab-http.log /tmp/testfile.txt
[2]+  Terminated              python3 -m http.server 8088 --directory /tmp > /tmp/lab-http.log 2>&1
tp@ahmed-abdelhedi:~$ ls /tmp/.lab-updater.sh
ls: cannot access '/tmp/.lab-updater.sh': No such file or directory
tp@ahmed-abdelhedi:~$
```

VII. THE TEMPORAL PROBLEM: WHAT DID WE MISS?

25+26:

```
tp@ahmed-abdelhedi:~$ sh -c 'echo short-lived-osquery-demo > /tmp/short-lived.txt'
tp@ahmed-abdelhedi:~$ sudo osqueryi
[sudo] password for tp:
Using a virtual database. Need help, type '.help'
osquery> SELECT pid, name, path, cmdline FROM processes WHERE cmdline LIKE '%short-live
dosquery-demo%';
osquery>
```

The query returns no result because the process terminates before the process-table snapshot is collected.

27:

```
tp@ahmed-abdelhedi:~$ sudo osqueryi
[sudo] password for tp:
Using a virtual database. Need help, type '.help'
osquery> SELECT pid, name, path, cmdline FROM processes WHERE cmdline LIKE '%short-live
dosquery-demo%';
osquery> SELECT path, filename, size, mode, uid, gid, mtime
    ...> FROM file
    ...> WHERE path = '/tmp/short-lived.txt';
+----------------------+----------------+------+------+------+------+------------+
| path                 | filename       | size | mode | uid  | gid  | mtime      |
+----------------------+----------------+------+------+------+------+------------+
| /tmp/short-lived.txt | short-lived.txt | 25   | 0664 | 1001 | 1001 | 1789319802 |
+----------------------+----------------+------+------+------+------+------------+
osquery>
```

Although the process is no longer observable, /tmp/short-lived.txt remains on disk and can still be detected through the file table.

VIII. FROM SNAPSHOTS TO EVENT-BASED DETECTION

28:

```
tp@ahmed-abdelhedi:~$ sudo cat /etc/osquery/osquery.flags
--config_path=/etc/osquery/osquery.conf
--logger_plugin=filesystem
--logger_path=/var/log/osquery
--disable_events=false
--disable_audit=false
--audit_allow_config=true
--audit_persist=true
--audit_allow_process_events=true
--events_expiry=3600
tp@ahmed-abdelhedi:~$ sudo cat /etc/osquery/osquery.conf
{
  "schedule": {
    "process_events_lab": {
      "query": "SELECT pid, parent, path, cmdline, uid, time FROM process_events;",
      "interval": 5
    }
  }
}
tp@ahmed-abdelhedi:~$
```

```
tp@ahmed-abdelhedi:~$ sudo systemctl status osqueryd --no-pager
● osqueryd.service - osquery daemon
     Loaded: loaded (/usr/lib/systemd/system/osqueryd.service; disabled; preset: enable
d)
     Active: active (running) since Sun 2026-09-13 18:34:31 CET; 6s ago
    Process: 45135 ExecStartPre=/bin/sh -c if [ ! -f $FLAG_FILE ]; then touch $FLAG_FIL
E; fi (code=exited, status=0/SUCCESS)
    Process: 45136 ExecStartPre=/bin/sh -c if [ -f $LOCAL_PIDFILE ]; then mv $LOCAL_PID
FILE $PIDFILE; fi (code=exited, status=0/SUCCESS)
   Main PID: 45138 (osqueryd)
      Tasks: 17 (limit: 4598)
     Memory: 24.6M (peak: 24.8M)
        CPU: 85ms
     CGroup: /system.slice/osqueryd.service
             ├─45138 /opt/osquery/bin/osqueryd --flagfile /etc/osquery/osquery.flags -…
             └─45141 /opt/osquery/bin/osqueryd

Sep 13 18:34:31 ahmed-abdelhedi systemd[1]: Starting osqueryd.service - osquery da…n...
Sep 13 18:34:31 ahmed-abdelhedi systemd[1]: Started osqueryd.service - osquery daemon.
Sep 13 18:34:31 ahmed-abdelhedi osqueryd[45138]: osqueryd started [version=5.23.1]
Sep 13 18:34:31 ahmed-abdelhedi osqueryd[45141]: I0913 18:34:31.549237 45141 datab…ate.
Sep 13 18:34:31 ahmed-abdelhedi osqueryd[45141]: I0913 18:34:31.553261 45141 event…tion
Sep 13 18:34:31 ahmed-abdelhedi osqueryd[45141]: I0913 18:34:31.553395 45141 event…tion
Sep 13 18:34:31 ahmed-abdelhedi osqueryd[45141]: I0913 18:34:31.553406 45141 event…tion
Sep 13 18:34:35 ahmed-abdelhedi osqueryd[45141]: I0913 18:34:35.694187 45170 query…_lab
Hint: Some lines were ellipsized, use -l to show in full.
tp@ahmed-abdelhedi:~$
```

Osqueryd is configured to use the Linux Audit publisher and the process_events table to collect process-execution events every five seconds.

29:

```
tp@ahmed-abdelhedi:~$ sh -c 'echo short-lived-osquery-demo > /tmp/short-lived.txt'
tp@ahmed-abdelhedi:~$ sudo osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT pid, name, path, cmdline
    ...> FROM processes
    ...> WHERE cmdline LIKE '%short-lived-osquery-demo%';
osquery>
```

```
tp@ahmed-abdelhedi:~$ sh -c 'echo short-lived-osquery-demo > /tmp/short-lived.txt'
tp@ahmed-abdelhedi:~$ sudo osqueryi
Using a virtual database. Need help, type '.help'
osquery> SELECT pid, name, path, cmdline
    ...> FROM processes
    ...> WHERE cmdline LIKE '%short-lived-osquery-demo%';
osquery> .exit
tp@ahmed-abdelhedi:~$ sudo grep -F 'short-lived-osquery-demo' \
/var/log/osquery/osqueryd.results.log | tail -n 3
{"name":"process_events_lab","hostIdentifier":"ahmed-abdelhedi","calendarTime":"Sun Sep
 13 17:39:05 2026 UTC","unixTime":1789321145,"epoch":0,"counter":4,"numerics":false,"co
lumns":{"cmdline":"sh -c echo short-lived-osquery-demo > /tmp/short-lived.txt","parent"
:"2880","path":"/usr/bin/dash","pid":"45188","time":"1789321143","uid":"1001"},"action"
:"added"}
tp@ahmed-abdelhedi:~$
```

From the collected event:

- Process ID (PID): 45188
- Parent process ID: 2880
- Executable path: /usr/bin/dash
- Command line: sh -c echo short-lived-osquery-demo > /tmp/short-lived.txt
- User/UID: 1001 (tp)
- Execution time: 1789321143 (Unix timestamp), corresponding to 13 September 2026 at 17:39:03 UTC.

30:

Event-based collection detects short-lived activity but generates more events, increases CPU and memory usage, requires additional storage, and needs a retention policy to control how long events are preserved.